JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY, NOIDA

B.TECH V SEMESTER

TITLE

Social Network Graph Analyzer

SYNOPSIS

DSA (25B17CI578)



]

Team Members	**Enrollment No.**
1. Vedansh Goel	2401020188
2. Kavya Singhal	2401020194
3. Mohita Vig	2401020202
4. Parigya Gupta	2401020207

Submitted to: Dr. Arpita Jadhav Bhatt

1. Problem Statement

The **Social Network Graph Analyzer** is a high-performance graph processing system designed to parse, store, and analyze large-scale social network data using **C++ and Data Structures & Algorithms (DSA)**. The system converts raw edge-list data into a memory-efficient **Compressed Sparse Row (CSR)** representation and applies graph traversal, searching, sorting, hashing, and connectivity algorithms for efficient network analysis.

The project focuses on implementing **efficient graph data structures, STL containers, pointers, dynamic memory allocation, recursion, queues, stacks, priority queues, hashing, and time-space complexity optimization** using C++.

2.Proposed approach

High-Performance Backend using C++

- The system will parse raw edge-list datasets and store the graph using **C++ and CSR (Compressed Sparse Row)** representation. C++ will be used for:
- File handling using ifstream/ofstream
- Dynamic memory allocation, pointers and references
- STL containers like vector, queue, stack, unordered_map, and priority_queue
- Sorting, searching, and graph traversal
- Time and space complexity optimization

CSR-Based Graph Representation

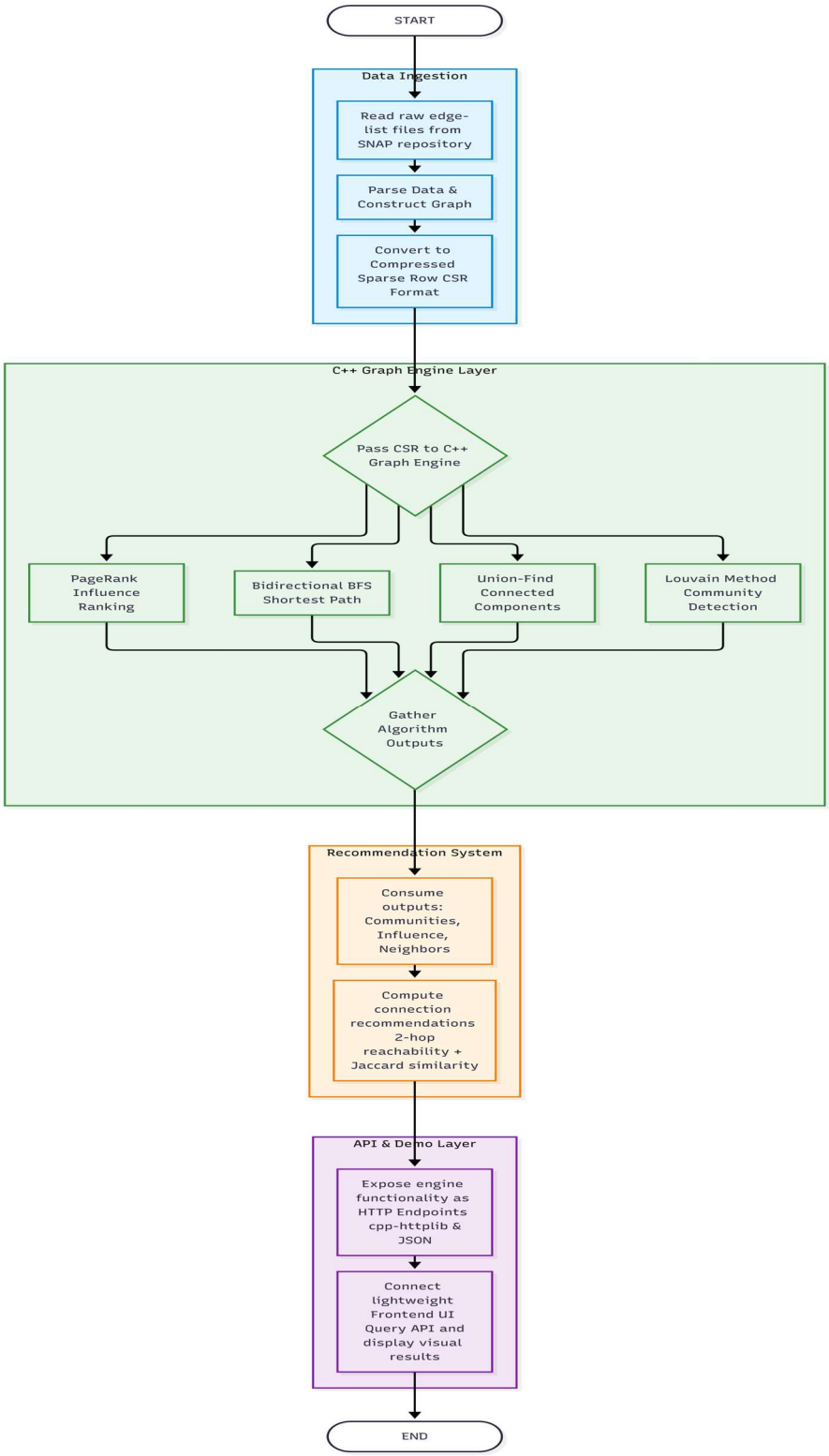
- CSR stores the graph using two contiguous arrays:
- **row_ptr** – starting position of each vertex's neighbors
- **col_idx** – stores adjacent vertices
- This reduces pointer overhead, saves memory, and provides efficient cache-friendly access compared to traditional linked structures.

Graph Processing

- The CSR graph will be processed using:
- **Bidirectional BFS** for shortest paths
- **BFS/DFS** for traversal
- **Union-Find** for connectivity
- **Iterative computation and sorting** for node ranking
- **Hashing** for fast lookup and duplicate checking
- **2-hop traversal** for finding potential connections
- **Degree and neighbor analysis**

Frontend UI & API

- The C++ backend will communicate with a web-based frontend through an **HTTP REST API using cpp-httplib**. The dashboard will display shortest paths, rankings, graph statistics, and algorithm results.



3. Features and concepts to be used

Core DSA and C++ Concepts

- Graph Representation: Compressed Sparse Row (CSR)
- Graph Traversal: BFS and DFS
- Shortest Path: Bidirectional BFS
- Connectivity: Union-Find / Disjoint Set Union with path compression
- Searching: Linear and hash-based searching
- Sorting: STL sorting and custom comparators
- Hashing: unordered_map and unordered_set
- Priority Queue: Maintaining ranked graph results
- Recursion: Recursive graph traversal where applicable
- Arrays and Vectors: Efficient storage and traversal of graph data
- Pointers and References: Efficient memory access
- Dynamic Memory Allocation: Runtime graph storage
- File Handling: Reading and processing large edge-list datasets
- Classes and Objects: Modular C++ implementation
- STL: vector, queue, stack, unordered_map, unordered_set, priority_queue
- Time & Space Complexity: Analysis and optimization of algorithms
- Cache-Friendly Memory Access: Efficient traversal using contiguous CSR arrays

Application Features

- Fast ingestion of large graph datasets from text files
- Memory-efficient CSR graph construction
- Interactive shortest-path finder using Bidirectional BFS
- BFS/DFS-based graph traversal
- Identification of connected components using Union-Find

- Ranked node lists using iterative computation and sorting
- Finding potential connections using 2-hop neighbor traversal
- Fast vertex and edge lookup using hashing
- Graph statistics such as degree and number of connections
- Responsive dark-mode dashboard
- C++ REST API for communication between backend and frontend

4. Teamwork & Planning

- **Vedansh Goel – Data Structures: Data ingestion, CSR graph construction, and memory optimization.**
- **Kavya Singhal – Graph Algorithms: Bidirectional BFS, BFS/DFS, and node-ranking implementation.**
- **Mohita Vig – Connectivity & Searching: Union-Find, 2-hop traversal, hashing, and neighbor analysis.**
- **Parigya Gupta – API & Frontend: C++ REST API using cpp-httplib and dashboard development.**

5.References

1. **Google Gemini, AI Assistant. [Online].**
   Available: https://gemini.google.com/ ***(Utilized for algorithmic logic assistance, debugging C++ code, and architecture brainstorming).***
2. **GitHub Open Source Repositories. [Online].**
   Available: https://github.com/ ***(Reference for open-source header-only libraries used in the project, such as cpp-httplib for the API and nlohmann/json for JSON parsing).***
3. **GeeksforGeeks, "Graph Data Structure And Algorithms." [Online].**
   Available: https://www.geeksforgeeks.org/graph-data-structure-and-algorithms/ ***(Reference for logic and pseudo-code for Bidirectional BFS and Union-Find algorithms).***